

Microcontroladores

Roteiro 6 — Timers e PWM

Raoni F. S. Teixeira · DENE/UFMT · 1 sessão · 1,0 ponto · grupos de 3

Objetivos desta sessão

Ao final desta sessão o grupo deve ser capaz de:

1. Configurar o Timer2 e os módulos CCP em modo PWM;
2. Calcular período e frequência do PWM a partir de PR2 e do prescaler;
3. Medir duty cycle e frequência no osciloscópio;
4. Distinguir o sinal no pino do microcontrolador do sinal na carga.

⚠ Atenção

Sessão com osciloscópio. É uma das duas aulas do semestre com instrumentação. Reserve tempo para as medidas — elas são o centro do roteiro, não um complemento.

1 Configuração de bancada

Configuração de bancada

CH1-7 (TEMP)	ON
CH2-1 (LCD)	ON
CH3-3 (AQUECEDOR)	ON
CH3-5 (VENTILADOR)	ON
CH3-4, CH3-6, CH3-7	OFF
SWITCHS	Todas em OFF

Osciloscópio: ponta de prova em 10×, **acoplamento DC**, escala vertical em 1 ou 2 V/div, filtro de largura de banda **desligado**. Terra em qualquer GND dos conectores de porta. Compense a ponta antes de começar.

Em acoplamento AC a onda quadrada vira dente de serra; em 100 mV/div o sinal sai da tela.

2 Fundamento

2.1 Meia potência com um pino digital

O pino só assume 0 V ou 5 V. Não existe valor intermediário.

A solução é **chavear rápido**: se o pino fica metade do tempo em 5 V e metade em 0 V, e o chaveamento é muito mais rápido que a resposta da carga, a carga se comporta como se recebesse metade da potência.

A fração de tempo em nível alto chama-se **duty cycle**.

Conceito central

O PWM não gera tensão intermediária. Ele explora o fato de que a carga — um motor com inércia mecânica, uma resistência com inércia térmica — não consegue responder a cada pulso individualmente e reage à média.

Se a carga fosse rápida o bastante para seguir os pulsos, o PWM não funcionaria.

2.2 Período do PWM

Com o Timer2 como base de tempo:

$$T_{\text{PWM}} = (\text{PR2} + 1) \times 4 \times T_{\text{osc}} \times \text{prescaler}$$

Com $F_{\text{osc}} = 16 \text{ MHz}$ e $\text{PR2} = 255$:

Prescaler	Período	Frequência
1:1	64,0 μs	15,6 kHz
1:4	256,0 μs	3,9 kHz
1:16	1024,0 μs	977 Hz

Tabela 1: Frequências de PWM disponíveis com $\text{PR2} = 255$.

Divergência do manual

O clock da CPU é 16 MHz, não 48 MHz.

Os config bits gravados junto com o bootloader definem uma árvore de clock em que o periférico USB recebe 48 MHz e a CPU recebe 16 MHz. Os `#pragma config` da aplicação são ignorados.

Isso foi descoberto por medição, não por leitura de documentação: uma onda quadrada programada para 300 ms mediu 375 ms no osciloscópio, e o fator de 1,25 revelou o clock verdadeiro.

6.1 Verifique a primeira linha da tabela refazendo a conta. Mostre o desenvolvimento.

2.3 Onde vai o duty

O valor de duty tem 10 bits, repartidos entre dois registradores: os 8 bits altos em `CCPR1L` e os 2 baixos em `CCP1CON<5:4>`.

Nesta sessão usaremos apenas os 8 bits altos, o que dá resolução suficiente e simplifica o código.

3 Parte 1 — Medidas no pino

Grave o firmware `r6_pwm_osciloscopio.c`. Controles: **INT0 (SW12)** avança o duty; **INT1 (SW13)** avança a frequência. Os LEDs mostram o duty em barra.

Tarefa

1.1 — Meça na saída do microcontrolador: conector PORTC, pino RC2.

Mantenha a frequência em 977 Hz e varie o duty.

Duty nominal	t_{alto}	Período	Duty medido
25%			
50%			
75%			

1.2 O período mudou quando você alterou o duty? Isso era esperado?

🔪 Previsão — registre ANTES de gravar

1.3 — Agora você vai mudar a **frequência**, mantendo o duty em 50%.

Antes de fazer: a fração de tempo em nível alto vai mudar? Justifique.

Tarefa

1.4 — Meça nas três frequências, com duty fixo em 50%.

Prescaler	Frequência medida	Duty medido
1:16		
1:4		
1:1		

1.5 Compare com a Tabela 1. Os valores batem? Se houver desvio, ele é sistemático ou aleatório?

Conceito central

Duty e frequência são **graus de liberdade independentes**. Um controla quanta potência chega

à carga; o outro controla a granularidade do chaveamento. Confundir os dois é erro comum em projeto de acionamento.

4 Parte 2 — Pino contra carga

Esta é a medida mais importante da sessão.

🔧 Previsão — registre ANTES de gravar

2.1 — Você vai medir simultaneamente o pino RC2 do PIC e a saída do driver ULN2803, que aciona a ventoinha.

Antes de ligar: as duas formas de onda serão iguais? Terão a mesma amplitude? A mesma fase?

Tarefa

2.2 — Configure os dois canais:

- **Canal 1:** RC2 no conector PORTC
- **Canal 2:** test point da ventoinha, após o ULN2803
- Duty em 50%, frequência em 977 Hz, CH3-5 em ON

Desenhe as duas formas de onda em escala, uma sobre a outra.

2.3 As duas ondas têm a mesma amplitude? Qual o nível de tensão de cada uma?

2.4 Estão em fase ou invertidas? Explique com base no funcionamento do ULN2803.

Conceito central

O ULN2803 tem saída **open-collector**: quando a entrada está em nível alto, o transistor conduz e puxa a saída para o terra. A carga fica entre a saída e os +12 V.

Isso significa que “pino do microcontrolador em nível alto” e “carga energizada” são afirmações diferentes — e a diferença aparece na tela.

Todo driver de potência introduz alguma transformação desse tipo. Conhecer a do seu driver é parte do projeto, não detalhe de implementação.

5 Parte 3 — Aplicação ao termostato

Tarefa

3.1 — Substitua o controle liga/desliga do Roteiro 5 por atuação proporcional:

```
erro    = setpoint - temperatura;          /* décimos de grau */  
esforco = (erro * KP) / ESCALA_GANH0;
```

com $KP = 96$ e $ESCALA_GANHO = 64$ como ponto de partida.

O esforço positivo aciona o aquecedor; negativo, a ventoinha. Sature em 0 e 255.

🔍 Previsão — registre ANTES de gravar

3.2 — Com controle proporcional, a temperatura vai estabilizar **exatamente** no setpoint?

Responda antes de rodar, e justifique em termos do que acontece com o esforço quando o erro diminui.

3.3 O que de fato aconteceu? Em que temperatura o sistema estabilizou?

3.4 Se houve diferença em relação ao setpoint, quanto foi? Aumente KP para 192 e repita. A diferença diminuiu? Algo piorou?

Observação

O comportamento observado em 3.3 e 3.4 é o assunto central do Roteiro 9. Guarde os números.

6 Teste de aceitação

- ☐ Tabelas 1.1 e 1.4 preenchidas com medidas do osciloscópio.
- ☐ Desenho 2.2 das duas formas de onda, com escalas anotadas.
- ☐ Termostato com atuação proporcional funcionando.

7 Entrega e critério

Peso	Item
0,2	Teste de aceitação aprovado
0,2	Cálculo 6.1 e comparação 1.5 com os valores medidos
0,2	Previsão 1.3 e previsão 2.1, registradas antes das medidas
0,2	Respostas 2.3 e 2.4 sobre o driver
0,2	Previsão 3.2 e respostas 3.3 e 3.4

Tabela 2: Distribuição do ponto do Roteiro 6.

8 Armadilhas frequentes

Sintoma	Causa provável
Onda com bordas arredondadas	Ponta de prova em 1× ou não compensada
Nenhum sinal no pino	Timer2 desligado, ou CCPxCON fora do modo PWM
Duty não muda	Escreveu em CCP1H no lugar de CCP1L
Sinal instável na tela	Trigger mal ajustado; use borda de subida no canal 1
Ruído no chaveamento da carga	Normal: é o motor. Registre como observação
Aquecedor sempre ligado	CCP2MX fora de ON: CCP2 não está em RC1

Tabela 3: Diagnóstico rápido do Roteiro 6.

9 Para a próxima sessão

Tarefa

Seu programa hoje usa `__delay_ms()` para esperar entre as leituras. Durante essa espera, o processador não faz absolutamente nada.

Traga escrito: se você precisasse, ao mesmo tempo, ler o sensor a cada 200 ms, atualizar o LCD a cada 300 ms e verificar um botão a cada 10 ms — como faria isso com `__delay_ms()`?